



SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science & Engineering
ASSESSMENT TOOL 1– CONCEPT MAPPING



CourseCode	CSA0314	CourseTitle	Data Structures
COAssessed	CO1	Assessment	Assessment Tool1:Concept Mapping
Weightage	35% of CIA	TotalMarks	100
CO4Statement	<i>Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity. (BL2,BL3)</i>		
Student Name	sk.md.suhail	Reg.No.	192525229
Section/Batch	Slot D	Date	05-10-2026

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts	
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established	
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure	
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence	
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand	

Code of Conduct:

I, sk.md.suhail (192525229), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

sk.md.suhail



Q32. A real-time online examination system processes student responses, evaluates answers, and stores results instantly. Construct a concept map linking Data Flow → Processing → Data Structure Selection → Access Speed → Performance. Analyze how the flow of data influences the choice of data structures and justify the most efficient design.



CONCEPT MAPPING

1. DATA FLOW

Exam pipeline:

- Login → Fetch questions → Submit answers → Evaluate → Store result

Characteristics:

- Thousands of concurrent students
- Continuous stream, FIFO arrival order
- Burst load near the deadline
- Read-heavy (questions) + write-heavy (answers)

2. PROCESSING

Operations performed:

- Authenticate student & validate session
- Match answer with key (search by question ID)
- Compute and update the score
- Rank / sort the results
- Insert result record + activity log

Demand:

- Fast search, insert and update per response

REAL-TIME ONLINE EXAMINATION SYSTEM

Data Flow → Processing → DS Selection → Access Speed → Performance

4. ACCESS SPEED

Time complexity:

- Array index access – $O(1)$
- Hash table lookup – $O(1)$ average
- Queue enqueue / dequeue – $O(1)$
- Heap insert / extract – $O(\log n)$
- Balanced BST search – $O(\log n)$
- Linked list search – $O(n)$ ✗ slow

3. DATA STRUCTURE SELECTION

- **Queue** – buffers incoming submissions (FIFO)
- **Hash Table** – studentID → session; questionID → key
- **Array** – answer sheet indexed by Q. no.
- **Heap** – live ranking / top-k scores
- **Linked List** – dynamic activity log
- **Stack** – back / undo navigation

5. PERFORMANCE

Outcomes:

- Low latency – instant evaluation & results
- Scales with number of concurrent users
- Efficient memory use – space $O(n)$
- High throughput, no bottlenecks

Most efficient design:

- Queue + Hash Table + Array + Heap
- $O(1)$ for submit, lookup and score update
- $O(\log n)$ only for ranking
- Avoids $O(n)$ scans of lists

ANALYSIS

1. How the flow of data influences the choice of data structure

In an online examination system the data never stops moving: thousands of students submit answers at the same time, every answer must be checked immediately, and the result must be stored without delay. The shape of this flow decides what each structure must do well. Data that arrives in a stream and must be processed in the same order needs a queue. Data that is looked up again and again by an ID needs a hash table or an array, where access time does not grow with the number of students. Data that must stay ordered by score needs a heap. Data that only grows, such as logs, needs a structure with cheap appends. Choosing a structure that does not match the access pattern, for example a linked list for lookups, turns an $O(1)$ step into an $O(n)$ scan and slows the whole pipeline when the load is highest.

2. Mapping of flow stage to data structure

Stage	Data-flow behaviour	Operation needed	Data structure chosen	Cost
Incoming responses	Continuous burst of submissions from many students; must be handled in arrival order	Insert / remove in order	Queue (buffer)	Enqueue / dequeue $O(1)$
Session & student lookup	Every request carries a student ID that must be found instantly	Search by key	Hash table (studentID $\rightarrow$ session)	$O(1)$ average
Answer evaluation	Each answer is compared with the key for that question number	Direct access by index	Array / hash table (questionID $\rightarrow$ key)	$O(1)$
Score update & ranking	Scores change constantly; toppers needed live	Insert and extract max	Max-heap	$O(\log n)$
Result & activity log	Records only grow; no searching during the exam	Append	Linked list / dynamic array	$O(1)$ append

3. Justification of the most efficient design

The most efficient design is a hybrid, not a single structure: **Queue + Hash Table + Array + Heap.**

Queue: absorbs the burst of submissions and keeps them in first-in-first-out order, so no response is lost or processed out of turn. Both operations take $O(1)$.

Hash table: finds a student session or an answer key in $O(1)$ on average, regardless of how many students are online. A sorted array or BST would cost $O(\log n)$, and a linked list $O(n)$.

Array: the answer sheet has a fixed number of questions, so storing answers by question number gives $O(1)$ direct access with the lowest memory overhead and good cache behaviour.

Heap: keeps the highest scores available at the top. Inserting a score costs $O(\log n)$, which is far cheaper than re-sorting all results in $O(n \log n)$ after every submission.

Overall, submitting, locating, evaluating and updating a score are all $O(1)$, and only live ranking pays $O(\log n)$. Memory use stays $O(n)$ for n students, and no step depends on scanning a list. This keeps latency low, lets throughput scale with the number of concurrent users, and returns results instantly.

Conclusion: the flow of data (burst arrival, key-based lookup, continuous score change) dictates the operations required; matching each operation to the structure with the best access speed gives the best overall performance.